Prof. Dr. Stefan Leutenegger
Teaching Assistants: Shi Chen, Cédric Le Gentil, Alex Brandes, and Yannick Burkhardt

Exercise 9: Probabilistic Occupancy Mapping

Session date: 05/05/2026

Task 1: 2D Occupancy Mapping of a Diff-Drive Robot (Python)

Study the implementation of the simulator in `E9_diff_drive_occ.py`. You can steer the robot using the arrow keys. The robot is equipped with N laser range sensors, centered at the robot's body frame $\underrightarrow{\mathcal{F}}_B$ origin and pointing into different directions in angular increments of $\Delta\alpha$. Each sensor with index $i = 0, \ldots, N-1$ is oriented at an angle α_i relative to the robot's x axis: $\alpha_i = -\Delta\alpha(N-1)/2 + i\Delta\alpha$. The simulator generates measurements $\tilde{z}_i$ containing Gaussian noise and a uniform outlier component in the interval $[0, d_{\max}]$. To flag an invalid measurement (e.g., due to specular reflection or excessive range), $\tilde{z}_i = d_{\max}$ is returned.

Since we are focusing on mapping here, we assume that poses are given; we use the simulated ground-truth poses $\mathbf{x} = [x_R, y_R, \theta_R]^T$, where (x_R, y_R) is the 2D position in the world frame $\underrightarrow{\mathcal{F}}_W$ and θ_R is the heading angle. You are now meant to implement the Log-Odds 2D occupancy mapping algorithm from the lecture. Note that the visualisation of the Log-Odds occupancy grid is already implemented.

i) Propose an inverse sensor model and draw it as a graph with occupancy probability $P(\cdot)$ on the y-axis and distance from the sensor on the x-axis (with the necessary parameters labelled). Also draw it in the form we will later need, where the y-axis shows the Log-Odds value $l(\cdot)$. Make sure to accommodate the fact that there are outliers, possible violations of the static-world assumption, as well as distance measurement noise.

ii) (Python) Now implement the update function `OccupancyMap.update(x, z)`, where `z` contains all measurements $\tilde{z}_i$ and $\mathbf{x}$ is the full state vector. Make sure to saturate the Log-Odds values at some minimum and maximum.

To simplify things, assume perfect laser measurement beams, i.e. no need to account for a cone-like measurement. To further simplify, assume that as soon as a cell is intersected by a ray, the respective Log-Odds occupancy is updated according to the inverse sensor model above, irrespective of how close to the cell centre the ray passes.

Note: you will need to consider efficiency to make the simulation run at interactive rates. Make sure to pass the 2D array only once (not N times) per update step. Also try to make use of an early-abort mechanism when a cell is beyond sensor range or outside the angular frustum of all laser beams.

Task 2: 2D LiDAR-Based Self-Localization using an ESDF Map (Python)

The robot navigates the ETH-logo maze, whose walls are encoded in a pre-built truncated *Euclidean Signed Distance Function* (ESDF) map $\Phi\colon \mathbb{R}^2 \to [-\tau, \tau]$. At any world point $\mathbf{p}$, $\Phi(\mathbf{p})$ equals the signed Euclidean distance to the nearest wall surface, clamped to $\pm\tau$: positive inside navigable corridors, negative inside solid walls, and exactly zero on a wall surface. The map also provides the spatial gradient $\nabla\Phi(\mathbf{p})$, accessible via `ESDFMap.query(p)`.

The robot carries a full-circle LiDAR with N beams. Beam i ($i = 0, \ldots, N-1$) is oriented at angle $\alpha_i = 2\pi i/N$ in the body frame $\underrightarrow{\mathcal{F}}_B$ and returns range z_i. As in Task 1, measurements include Gaussian noise and an outlier component; $\tilde{z}_i = d_{\max}$ flags an invalid beam. Note that, unlike a real rotating LiDAR, all beams are modelled as a synchronised snapshot captured simultaneously, and the pose correction is applied every few frames to simplify the problem. The robot state is $\mathbf{x} = [x_R,\, y_R,\, \theta_R]^\top$, where (x_R, y_R) is the 2D position in the world frame $\underrightarrow{\mathcal{F}}_W$ and θ_R is the heading angle. For a given pose $\mathbf{x}$, the world-frame endpoint of beam i is

$$\mathbf{p}_i(\mathbf{x}) = \begin{bmatrix} x_R + z_i \cos(\theta_R + \alpha_i) \\ y_R + z_i \sin(\theta_R + \alpha_i) \end{bmatrix}.$$

i) (Python) Open `E9_diff_drive_esdf.py` and run the simulation. Try to navigate the robot to the goal using the arrow keys. The displayed blue robot shows the estimated pose, which relies on odometry integration only (no LiDAR correction yet). What do you observe?

   Hint: Press `D` to toggle debug mode.

ii) Define the error $e_i(\mathbf{x})$ for beam i with valid range $z_i < d_{\max}$, and write the cost function $c(\mathbf{x})$.

   Hint: Recall that $\tilde{z}_i = d_{\max}$ flags an invalid measurement.

iii) Derive the Jacobian $\mathbf{E}_i = \frac{\partial e_i}{\partial \mathbf{x}} \in \mathbb{R}^{1\times 3}$.

   Hint: First compute $\frac{\partial \mathbf{p}_i}{\partial \mathbf{x}} \in \mathbb{R}^{2\times 3}$ entry by entry, then combine with $\nabla\Phi(\mathbf{p}_i)$ using the chain rule.

iv) Write down the normal equations $\mathbf{A}\,\Delta\mathbf{x} = \mathbf{b}$ from which we can solve for the update $\Delta\mathbf{x}$.

v) (Python) Using the Gauss-Newton optimisation scheme, implement `PoseEstimator.estimate(z, x0)` in `E9_diff_drive_esdf.py`. Validate your implementation by navigating the robot successfully to the goal.

   Hint: You may reuse the Gauss-Newton solver you implemented in Exercise 6, or simply use `numpy.linalg.solve()`.

vi) (Python) Dial down the maximum LiDAR range `MAX_RANGE` to an extremely low 2.0 in `E9_diff_drive_esdf.py` and try the navigation again. What happens to the matrix $\mathbf{A}$ in the normal equations and why does it happen? How could it be solved?